ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

СОГЛАСОВАНО	УТВЕРЖДЕНО
Руководитель направления «ПАО Московская Биржа»	Академический руководитель образовательной программы «Программная инженерия», старший преподаватель департамента программной инженерии
_____ Е. А. Кондратьев	_____ Н. А. Павлочев
«___» _____ 2026 г.	«___» _____ 2026 г.

Подп. и дата	
Инв. № дубл.	
Взам. инв. №	
Подп. и дата	
Инв. № подп.	

РАЗРАБОТКА РИСК-КАЛЬКУЛЯТОРА ДЛЯ РЫНКА СПФИ МОСКОВСКОЙ БИРЖИ.

Техническое задание. Общая часть

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.05.01-01 ТЗ 01-1-ЛУ

Исполнители:
Студенты группы БПИ235
_____ / К. В. Воякин
А. О. Сазонов /
«___» _____ 2026 г.

2026

УТВЕРЖДЕН

RU.17701729.05.01-01 ТЗ 01-1-ЛУ

РАЗРАБОТКА РИСК-КАЛЬКУЛЯТОРА ДЛЯ РЫНКА СПФИ МОСКОВСКОЙ БИРЖИ.

Техническое задание. Общая часть

RU.17701729.05.01-01 ТЗ 01-1

Листов 9

Подп. и дата	
Инв. № дубл.	
Взам. инв. №	
Подп. и дата	
Инв. № подп.	

2026

АННОТАЦИЯ

Настоящее техническое задание устанавливает требования к командной разработке риск-калькулятора для рынка СПФИ Московской биржи. Программа должна обеспечивать импорт портфеля, проверку входных данных, подключение market data bundle, расчет риск-показателей, стресс-анализ, контроль лимитов и экспорт отчетности.

В документе указаны основания и назначение разработки, требования к функциональным характеристикам, надежности, условиям эксплуатации, программной документации, стадиям разработки и порядку приемки.

Документ подготовлен по текущему состоянию проекта: Python/FastAPI backend, React/TypeScript/Vite frontend, CLI, тесты и runtime audit checks.

СОДЕРЖАНИЕ

1. ТЕХНИЧЕСКОЕ ЗАДАНИЕ 4
1.1. Введение 4
1.2. Основания для разработки 4
1.3. Назначение разработки 4
1.4. Требования к функциональным характеристикам 4
1.4.1. Импорт и валидация входных данных 4
1.4.2. Поддерживаемые инструменты и модели оценки 5
1.4.3. Рыночные данные 5
1.4.4. Расчет риск-показателей 5
1.4.5. API и CLI 6
1.4.6. Пользовательский интерфейс 6
1.4.7. Экспорт результатов 6
1.5. Требования к надежности 6
1.6. Требования к условиям эксплуатации 7
1.7. Требования к программной документации 7
1.8. Стадии и этапы разработки 7
1.9. Порядок контроля и приемки 8
2. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ 9

1. ТЕХНИЧЕСКОЕ ЗАДАНИЕ

1.1. Введение

Настоящее техническое задание распространяется на разработку web-платформы Risk Calculator for MOEX SPFI. Программа предназначена для импорта портфеля производных финансовых инструментов, загрузки рыночных данных, расчета риск-метрик, анализа стресс-сценариев, контроля лимитов и экспорта результатов.

Разработка ведется как командный проект. Программный продукт включает backend на Python/FastAPI, frontend на React/TypeScript/Vite, CLI для пакетного запуска, набор тестов и runtime audit checks.

1.2. Основания для разработки

Основанием для разработки является задание на курсовой проект по образовательной программе «Программная инженерия». Тема разработки: риск-калькулятор для рынка стандартизированных производных финансовых инструментов Московской биржи.

1.3. Назначение разработки

Функциональное назначение программы – автоматизация расчета и анализа риск-показателей портфеля. Программа должна позволять пользователю загрузить данные портфеля, проверить корректность данных, подключить market data bundle, задать параметры расчета, получить результаты на dashboard, выполнить стресс-анализ, проверить лимиты и выгрузить отчет.

Эксплуатационное назначение программы – использование в учебном процессе как прототипа risk analytics platform. Система показывает полный процесс обработки данных от импорта до расчета VaR/ES, LC VaR, греков, стрессов, лимитов и выгрузки отчетности.

1.4. Требования к функциональным характеристикам

1.4.1. Импорт и валидация входных данных

Программа должна поддерживать импорт портфеля из CSV, JSON, XLS и XLSX. Frontend должен поддерживать drag-and-drop, вставку текста, загрузку демо-портфеля и предпросмотр позиций. Для CSV должны обрабатываться кодировки UTF-8 и Windows-1251.

Программа должна валидировать обязательные поля позиции: тип инструмента, идентификатор, направление, цену базового актива, strike, волатильность, дату оценки, дату погашения, ставку, валюту и количество. Для опционов волатильность должна быть положительной; для других инструментов не должна быть отрицательной. Валюта должна соответствовать ISO 4217.

Программа должна формировать журнал валидации. В frontend журнал должен отображаться на странице `/validate`, группироваться по типам замечаний и выгружаться в `validation_log.json`.

1.4.2. Поддерживаемые инструменты и модели оценки

Программа должна поддерживать инструменты типов `option`, `forward`, `swap_ir`. Для опционов должны быть доступны европейские и американские стили, типы call/put и модели Black-Scholes, биномиальная модель и Monte Carlo. Для форвардов должна быть доступна переоценка по рыночному контексту. Для процентных инструментов должна быть доступна оценка IRS/OIS/XCCY/basis swap.

Программа должна рассчитывать портфельную стоимость в базовой валюте. Для мультивалютного портфеля должны использоваться переданные FX rates или FX spot из market data. При отсутствии FX-покрытия расчет должен завершаться диагностируемой ошибкой, а не выдавать агрегированные метрики на некорректной базе.

1.4.3. Рыночные данные

Программа должна поддерживать Excel market data bundle со следующими типами файлов: `curveDiscount.xlsx`, `curveForward.xlsx`, `fixing.xlsx`, `calibrationInstrument*.xlsx`, `RC_*.xlsx`. Backend должен проверять типы файлов, размер загрузки, лимит строк XLSX, корректность структуры и полноту данных для расчета.

Программа должна поддерживать upload-сессии market data через API. Должны быть доступны загрузка файлов пользователем, загрузка default bundle из `Datasets/Данные для работы` и live sync через endpoint `/market-data/sync-live`.

1.4.4. Расчет риск-показателей

Программа должна рассчитывать:

- базовую стоимость портфеля;
- матрицу сценарного PnL;
- historical VaR и historical ES;
- parametric VaR и parametric ES;
- Cornish-Fisher tail adjustment для параметрического хвоста;
- liquidity-adjusted VaR и breakdown ликвидностной надбавки;
- греки Delta, Gamma, Vega, Theta, Rho и DV01;
- stress results и worst stress;
- проверку лимитов;

- корреляции по позициям с ограничением размера матрицы;
- economic capital, initial margin, variation margin;
- buckets по валютам.

1.4.5. API и CLI

Backend должен предоставлять FastAPI endpoint'ы:

- `POST /metrics` – расчет портфеля;
- `POST /market-data/session` – создание сессии загрузки;
- `POST /market-data/upload` – загрузка market data файлов;
- `POST /market-data/load-default` – загрузка bundled market data;
- `POST /market-data/sync-live` – live sync рыночных данных;
- `GET /market-data/health` – проверка свежести market data session;
- `GET /market-data/{session_id}` – summary сессии;
- `GET /scenarios` – каталог сценариев;
- `GET /limits` – каталог лимитов;
- `GET /health` – health-check сервиса.

CLI должен поддерживать запуск расчета из командной строки, передачу портфеля, сценариев, лимитов, каталога market data, режима проверки market data и отключение отдельных групп расчетов.

1.4.6. Пользовательский интерфейс

Frontend должен реализовывать workflow Import -> Validate -> MarketData -> Configure -> Results -> Stress -> Limits -> Export. Недоступные шаги должны блокироваться маршрутизатором до завершения предыдущих этапов.

Пользовательский интерфейс должен включать страницы `/import`, `/validate`, `/market`, `/configure`, `/dashboard`, `/stress`, `/limits`, `/export`, `/portfolio`, `/help`, `/ui-demo`, `/hedge`, `/plan-b`. Основной сценарий должен выполняться через Vite proxy `/api` к FastAPI backend на `127.0.0.1:8000`.

1.4.7. Экспорт результатов

Программа должна поддерживать экспорт результатов расчета в CSV, Excel и JSON. CLI должен формировать `csv/`, `report.xlsx`, `report.json`, `pnl_hist.png`. Frontend должен формировать `risk_report.xlsx` и `risk_report.json`.

1.5. Требования к надежности

Программа должна диагностировать некорректные входные данные до расчета или на ранних этапах расчета. Backend должен возвращать структурированные ошибки API с request id или trace

id. Загрузка файлов должна иметь защиту от path traversal, непредусмотренных имен файлов, превышения размера и некорректных XLSX.

Расчеты должны быть воспроизводимыми. Monte Carlo должен использовать фиксированный seed. Параметры расчета – уровень доверия, горизонт, tail model, базовая валюта, liquidity model и включенные метрики – должны явно передаваться в API payload.

1.6. Требования к условиям эксплуатации

Для локального запуска требуется Python 3.12, Node.js 22, npm 10, браузер с поддержкой современного JavaScript. Backend-зависимости перечислены в `backend/requirements.txt`, frontend-зависимости – в `frontend/package.json`.

Единый запуск должен выполняться командой:

```bash
bash run_all.sh
```

После успешного запуска backend должен быть доступен по адресу `http://127.0.0.1:8000`, frontend – по адресу `http://127.0.0.1:5173`.

1.7. Требования к программной документации

В состав программной документации входят пояснительная записка, техническое задание, руководство оператора, программа и методика испытаний, текст программы, ссылка на репозиторий и дополнительные документы при необходимости. Документы подготавливаются в формате Typst, чтобы их можно было редактировать и собирать в PDF.

1.8. Стадии и этапы разработки

Разработка выполняется по план-графику 17.11.2025–10.04.2026.

- 17.11.2025–28.11.2025 – сбор требований, определение состава документов, ролей командного проекта и основного workflow Import -> Validate -> MarketData -> Configure -> Results -> Stress -> Limits -> Export.
- 01.12.2025–19.12.2025 – реализация моделей входных данных, загрузки портфеля, сценариев, лимитов, первичной валидации и структуры market data bundle.
- 22.12.2025–30.01.2026 – реализация расчетного ядра: pricing для options/forwards/swaps, Greeks, VaR/ES, LC VaR, мультивалютная агрегация и PnL distribution.
- 02.02.2026–20.02.2026 – реализация стресс-тестирования, библиотеки сценариев, лимитов, correlation matrix, economic capital, initial margin и variation margin.
- 23.02.2026–13.03.2026 – реализация frontend workflow, экранов импорта, проверки данных, market data, настройки расчета, dashboard, stress, limits и export.

- 16.03.2026–27.03.2026 – финальная интеграция FastAPI, CLI, frontend, bundled market data, Vite proxy и экспортных отчетов.
- 30.03.2026–10.04.2026 – автоматизированные испытания, runtime audit checks, проверка `run_all.sh` и подготовка сдачной документации.

1.9. Порядок контроля и приемки

Приемка программы выполняется по результатам автоматизированных проверок и ручного прохождения основного пользовательского сценария. Обязательные проверки: `pytest` для backend, `npm test` для frontend, `bash run_all.sh`, проверка `/health`, проверка доступности frontend, выборочные runtime audit scripts.

Программа считается принятой, если backend-тесты и frontend-тесты проходят без ошибок, единый запуск поднимает API и UI, valid portfolio возвращает расчетные метрики, invalid portfolio возвращает диагностируемую ошибку, неполный market data bundle блокируется, экспорт результатов формируется.

2. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. ГОСТ 19.101-77. Единая система программной документации. Виды программ и программных документов.
2. ГОСТ 19.102-77. Единая система программной документации. Стадии разработки.
3. ГОСТ 19.201-78. Техническое задание. Требования к содержанию и оформлению.

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

Лист регистрации изменений									
Номера листов (страниц)					Всего листов (страниц в докум.)	№ документа	Входящий № сопроводительного докум. и дата	Подп.	Дата
Изм.	Измененных	Замененных	Новых	Аннулированных					